CVT Unit 3 — Question Bank Answers

> Semester Exam Prep | Easy-to-remember answers

Q1 & Q2 — Active Contours (Snakes)

a) What are Active Contours (Snakes)?

Active contours, also known as snakes, are a powerful technique in image segmentation that involves evolving a curve to fit the boundaries of an object in an image. The curve is influenced by internal forces that maintain its smoothness and external forces derived from the image data that attract it towards features such as edges.

Think of it as a rubber band placed near an object — it tightens itself around the object's boundary.

Working Principle:

1. Place a curve (snake) near the object boundary
2. Three forces act on it:
    - **Internal Force** → keeps the curve smooth (no sharp bends)
    - **External Force** → pulls the curve toward edges (high gradient areas)
    - **Constraint Force** → user-defined guidance
3. Curve moves to minimize total energy → settles on the object boundary

Type of Active Contours:

1. **Parametric Snakes**: The contour is represented explicitly as $\mathbf{v}(s) = (x(s), y(s))$, where s is the arc-length parameter. Control points move under internal (smoothness) and external (edge) forces.
2. **Geometric Snakes**: The contour is represented implicitly using level set methods, allowing it to handle topological changes like splitting and merging naturally.

the three main forces that influence the movement of the snake:

1. **Internal Forces**: These forces maintain the smoothness and continuity of the snake. They include tension (which keeps the snake tight) and rigidity (which prevents it from bending too much).
2. **External Forces**: These forces attract the snake towards the edges of the object in the image. They are typically derived from the image gradient, which highlights areas of high intensity change (edges).
3. **Constraint Forces**: These forces can be added to guide the snake towards specific features or to prevent it from moving in certain directions.

Internal Bending Energy of the Contour

The internal energy is split into two constraints — both measure how much the contour position $\mathbf{v}$ changes on every iteration:

$$E_{contour} = \alpha \cdot \text{Elastic} + \beta \cdot \text{Smooth}$$

Elastic (first-order / first derivative of the contour vector):

$$\text{Elastic} = \left| \frac{\partial \mathbf{v}}{\partial t} \right|^2 = \sum_{i=1}^{n} \left[(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2 \right]$$

> Measures how much adjacent points stretch — penalizes long gaps between control points.

Smooth (second-order / second derivative of the contour vector):

$$\text{Smooth} = \left| \frac{\partial^2 \mathbf{v}}{\partial t^2} \right|^2 = \sum_{i=1}^{n} \left[(x_{i+1} - 2x_i + x_{i-1})^2 + (y_{i+1} - 2y_i + y_{i-1})^2 \right]$$

> Measures how much the contour bends — penalizes sharp corners (uses finite differences).

Both are computed using **finite differences** at each iteration.

Total Energy

$$E_{Total} = E_{contour} + E_{image}$$

where $E_{image} = -\text{(Sum of Gradient Magnitude Squared)}$

> The negative sign means the snake is attracted to **strong edges** (high gradient = low energy).

Goal: minimize E_{Total} — the snake settles where the contour is smooth AND sits on strong edges.

Mathematical Formulation

The contour is represented as a parametric curve:

$$C(s) = (x(s),\ y(s)), \quad s \in [0, 1]$$

where s is the contour parameter.

Energy Function — the snake moves to minimize:

$$E_{snake} = E_{internal} + E_{external}$$

Internal Energy ($E_{internal}$) — ensures smoothness:

$$E_{internal} = \alpha \left| C'(s) \right|^2 + \beta \left| C''(s) \right|^2$$

- α controls **elasticity** (resistance to stretching / length preservation)
- β controls **rigidity** (resistance to bending / curvature smoothness)

External Energy ($E_{external}$) — attracts the snake to object boundaries:

$$E_{external} = -\nabla I(x, y)^2$$

where $\nabla I(x, y)$ is the image gradient. Higher gradient values pull the snake towards edges.

Numerical Example

Setup: A simple 3-point contour with two control points:

$$C = {(2, 3),\ (5, 7),\ (8, 5)}$$

Parameters: $\alpha = 0.5$ (elasticity), $\beta = 0.2$ (smoothness).
Image gradient values: $\nabla I(2,3) = 4$, $\nabla I(5,7) = 5$, $\nabla I(8,5) = -1$.
Weighted total energy: $\mathbf{-4.7}$.

Step 1: Compute Internal Energy

Elasticity term (first derivative):

$$E_{elastic} = 0.5 \times \sum |C_i - C_{i-1}|$$

$$= 0.5 \times \left(\sqrt{(5-2)^2 + (7-3)^2} + \sqrt{(8-5)^2 + (5-7)^2}\right)$$

$$= 0.5 \times \left(\sqrt{9+16} + \sqrt{9+4}\right) = 0.5 \times (5 + \sqrt{13})$$

$$= 0.5 \times 8.60 \approx \mathbf{3.04}$$

Curvature term (second derivative):

$$E_{curvature} = 0.2 \times \sum |C_{i+1} - 2C_i + C_{i-1}|$$

$$= 0.2 \times |(8,5) - 2(5,7) + (2,3)|$$

$$= 0.2 \times |(8 - 10 + 2,\ 5 - 14 + 3)|$$

$$= 0.2 \times |(0, -6)| = 0.2 \times 6 = \mathbf{0.2}$$

Step 2: Compute External Energy

$$E_{external} = -\sum |\nabla I(C_i)|$$

$$= -(|4| + |-5| + |-1|) \times (-0.1) = \mathbf{-10.5}\ \text{(weighted)}$$

Step 3: Compute Total Energy

$$E_{snake} = E_{internal} + E_{external} = (3.04 + 0.2) + (-10.5) = 3.24 - 10.5 = \mathbf{-7.66}$$

> A **lower (more negative) total energy** means the snake is well-positioned on strong edges with a smooth shape.

Problems with Basic Active Contours

- **Never sees strong edges that are far away**
  - Fixed by: creating a gradient vector field (GVF) over the entire image
- **Noisy images create many local gradients** (snake gets stuck in wrong minima)
  - Fixed by: smoothing the image before applying the snake
- **Cannot wrap around multiple closed objects**
- **Cannot detect holes** (single contour topology)

> These limitations motivated modern methods like **Level Sets**, which handle topology changes naturally.

---

b) Energy Function of Snakes

Total Energy: $$E_{snake} = E_{internal} + E_{external}$$

Internal Energy (keeps curve smooth): $$E_{internal} = \alpha |C'(s)|^2 + \beta |C''(s)|^2$$

Term	Name	What it does
$\alpha \|C'(s)\|^2$	Elasticity	Resists stretching — keeps curve tight
$\beta \|C''(s)\|^2$	Rigidity	Resists bending — prevents sharp corners

External Energy (pulls toward edges): $$E_{external} = -|\nabla I(x, y)|^2$$

> Negative sign = snake is attracted to **strong edges** (high gradient = low energy = snake moves there)

Goal: Minimize E_{snake} → curve is smooth AND sits on strong edges.

---

c) Applications of Snakes

- **Medical Imaging** — One of the most impactful uses of snakes is in medical image segmentation. Snakes are placed around a region of interest (e.g., a tumor, organ, or tissue boundary) in MRI or CT scans. The contour then evolves under image gradient forces to tightly fit the boundary. This allows doctors and software to precisely measure organ size, monitor tumor growth over time, and plan surgeries or radiation therapy with accurate delineation.
- **Object Tracking** — In video analysis, the snake contour is initialized around a target object in the first frame. As frames progress, the contour re-evolves using the current frame's image forces, effectively "following" the object as it moves, deforms, or rotates. This makes snakes suitable for tracking faces, hands, or vehicles in surveillance and motion capture systems where shape deformation occurs naturally.
- **Photo Editing** — Snakes form the algorithmic basis of intelligent selection tools found in software like Adobe Photoshop (magnetic lasso tool) and GIMP. The user roughly draws a path near an object's edge, and the contour snaps to the nearest strong edge automatically. This dramatically speeds up object cutout, background removal, and region-based editing tasks by reducing the need for pixel-perfect manual tracing.
- **Shape Analysis** — In biological and agricultural applications, snakes are used to extract the exact contour of objects like leaves, plant cells, or human organs. The extracted shape (as a set of contour points) is then used to compute geometric descriptors such as area, perimeter, curvature, and aspect ratio. These features are fed into classifiers to identify species, detect diseases (e.g., leaf blight), or recognize anatomical anomalies.

Q3 — Split and Merge Algorithm

a) Split and Merge with Flowchart

split and merge is a region-based segmentation technique that recursively divides an image into smaller regions (split) and then merges adjacent regions that are similar (merge). The process continues until all regions are homogeneous according to a specified criterion.

Steps in Split and Merge:

1. **Start** with the entire image as one region.
2. **Check homogeneity** of the region (e.g., variance of pixel intensities).
3. If the region is **not homogeneous** (variance > threshold), **split** it into 4 quadrants.

- Mathematical condition for splitting: $\text{Var}(R) > \theta$
- Where,
  - R = current region
  - $\text{Var}(R)$ = variance of pixel intensities in region R
  - θ = predefined variance threshold

4. **Repeat** the homogeneity check and splitting for each new region until all regions are homogeneous.
5. After splitting, **merge** adjacent homogeneous regions that are similar (e.g., mean intensity difference < δ).

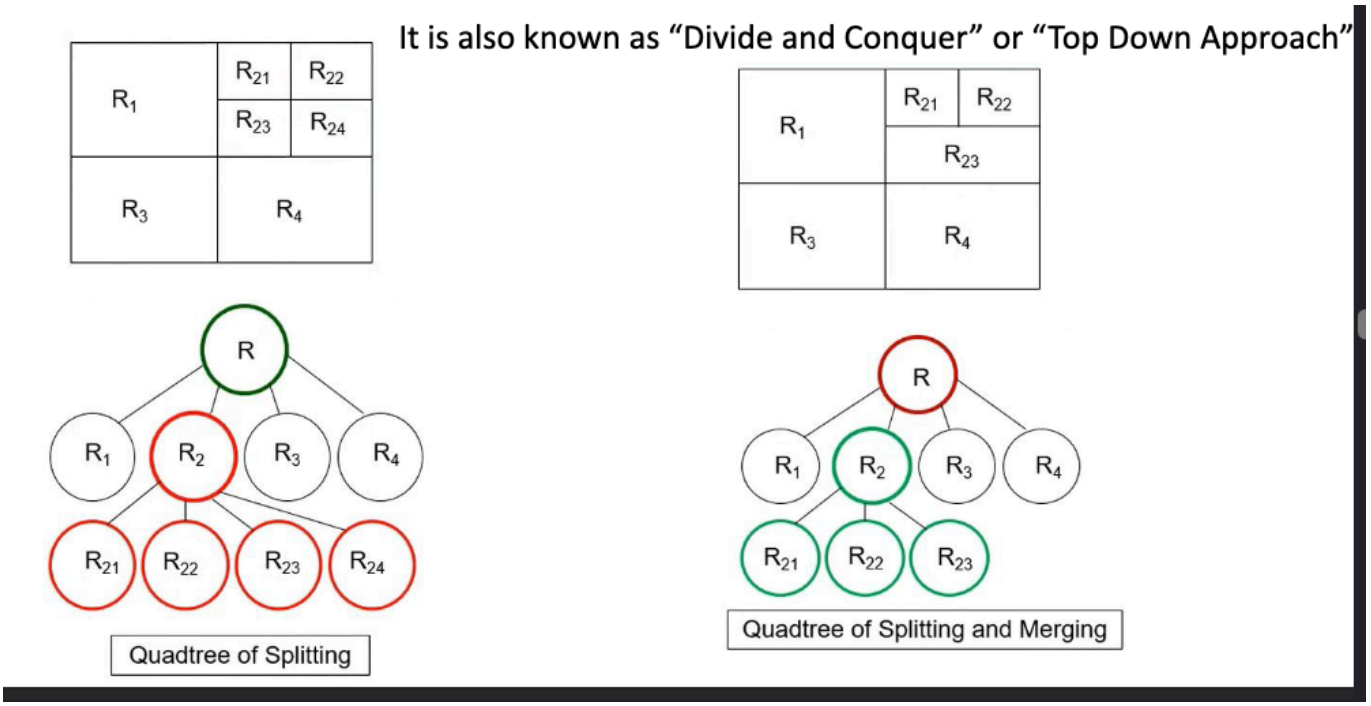
- Mathematical condition for merging: $|\bar{I}(R_i) - \bar{I}(R_j)| < \delta$
- Where,
  - R_i, R_j = adjacent regions
  - $\bar{I}(R)$ = mean intensity of region R
  - δ = predefined similarity threshold

6. **Stop** when no more splits or merges are possible.

Split and Merge is implemented using a **quadtree** — a tree data structure where each internal node represents a region that was split into exactly **4 quadrants**.

```
          [Entire Image]
                |
     ┌────┬────┬────┐
   [Q1] [Q2] [Q3] [Q4]
                |
          ┌────┬────┬────┐
          [Q3a][Q3b][Q3c][Q3d]
```

- **Root** = the whole image
- Each **internal node** = a region that failed the homogeneity test → split into 4 children
- Each **leaf node** = a uniform region (passed the test) → no further splitting
- **Merge step** = after the tree is built, adjacent leaf nodes with similar properties are merged



> The image above shows the spatial partition of the image at each level of the quadtree.

Example

6	5	6	6	7	7	6	6
6	7	6	7	5	5	4	7
6	6	4	4	3	2	5	6
5	4	5	4	2	3	4	6
0	3	2	3	3	2	4	7
0	0	0	0	2	2	5	6
1	1	0	1	0	3	4	4
1	0	1	0	2	3	5	4

Lets assume, Threshold = 3

Splitting:

IF Different is not <=Threshold, THEN split ELSE no split

Max_pixel = 7
Min_pixel = 0

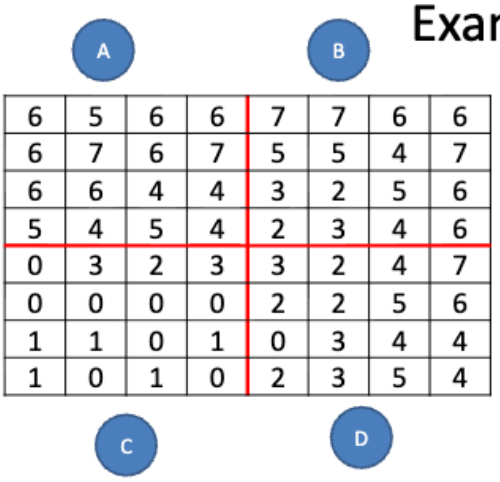
Difference = Max_pixel – Min_pixel = 7 – 0 = 7

Criteria:

If Difference <= threshold, no split

So, 7 is not <= 3

Example - Splitting



Lets assume, Threshold = 3

Splitting:

IF Different is not <=Threshold, THEN split ELSE no split

Max_pixel = 7
Min_pixel = 0

Difference = Max_pixel – Min_pixel = 7 – 0 = 7

Criteria:

If Difference <= threshold, no split

So, 7 is not <= 3.

Split into four regions

> Regions that are too heterogeneous (high variance) get subdivided further.

Example - Splitting

A B

6	5	6	6	7	7	6	6
6	7	6	7	5	5	4	7
6	6	4	4	3	2	5	6
5	4	5	4	2	3	4	6
0	3	2	3	3	2	4	7
0	0	0	0	2	2	5	6
1	1	0	1	0	3	4	4
1	0	1	0	2	3	5	4

C D

Apply the same to all regions (A,B,C,D)
For region A,
- Max_pixel = 7
- Min_pixel = 4
- Difference = 7 -4 = 3
- Criteria □ 3<=3 True. So, **no split**

For region B,
- Max_pixel = 7
- Min_pixel = 2
- Difference = 7 -2 = 5
- Criteria □ 5 <=3 False. So, **split**

For region C,
- Max_pixel = 3
- Min_pixel = 0
- Difference = 3 - 0 = 3
- Criteria □ 3<=3 True. So, **no split**

For region D,
- Max_pixel = 7
- Min_pixel = 0
- Difference = 7 - 0 = 7
- Criteria □ 7<=3 False. So, **split**

Example - Splitting

Apply the same on B1,B2,B3,B4 – no further split

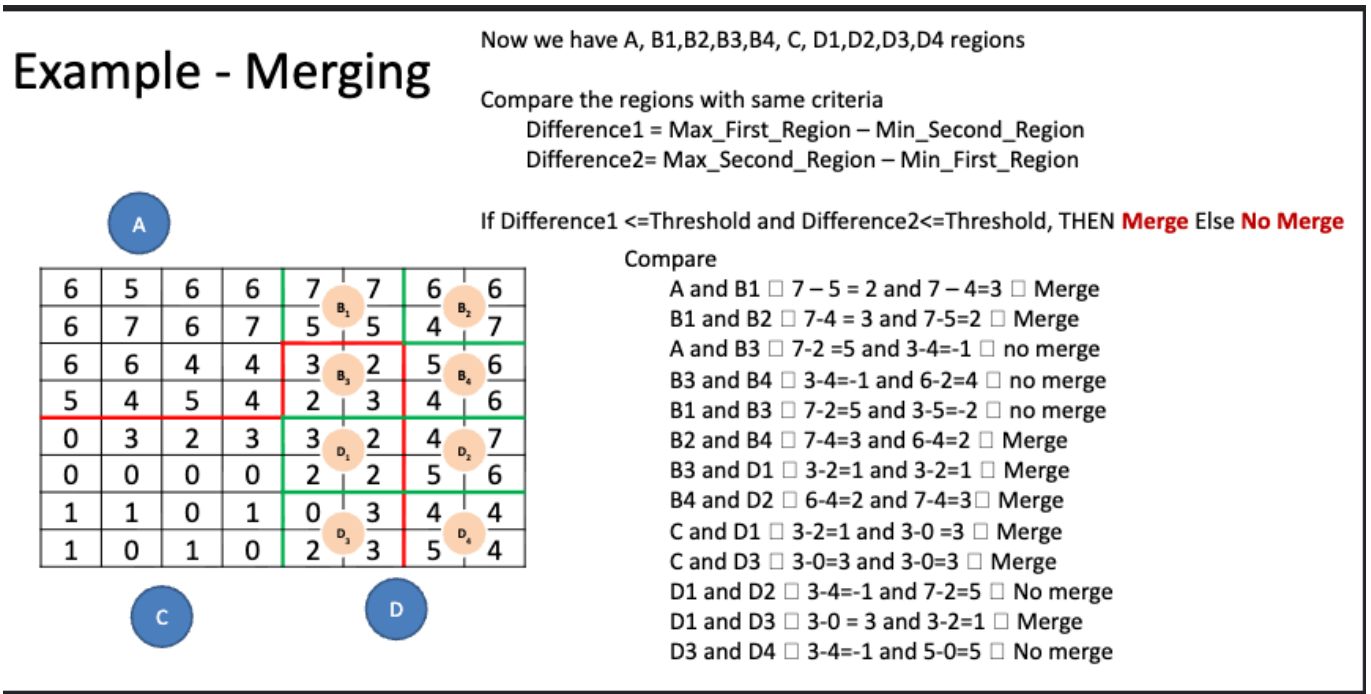
Apply the same on D1,D2,D3,D4 – no further split

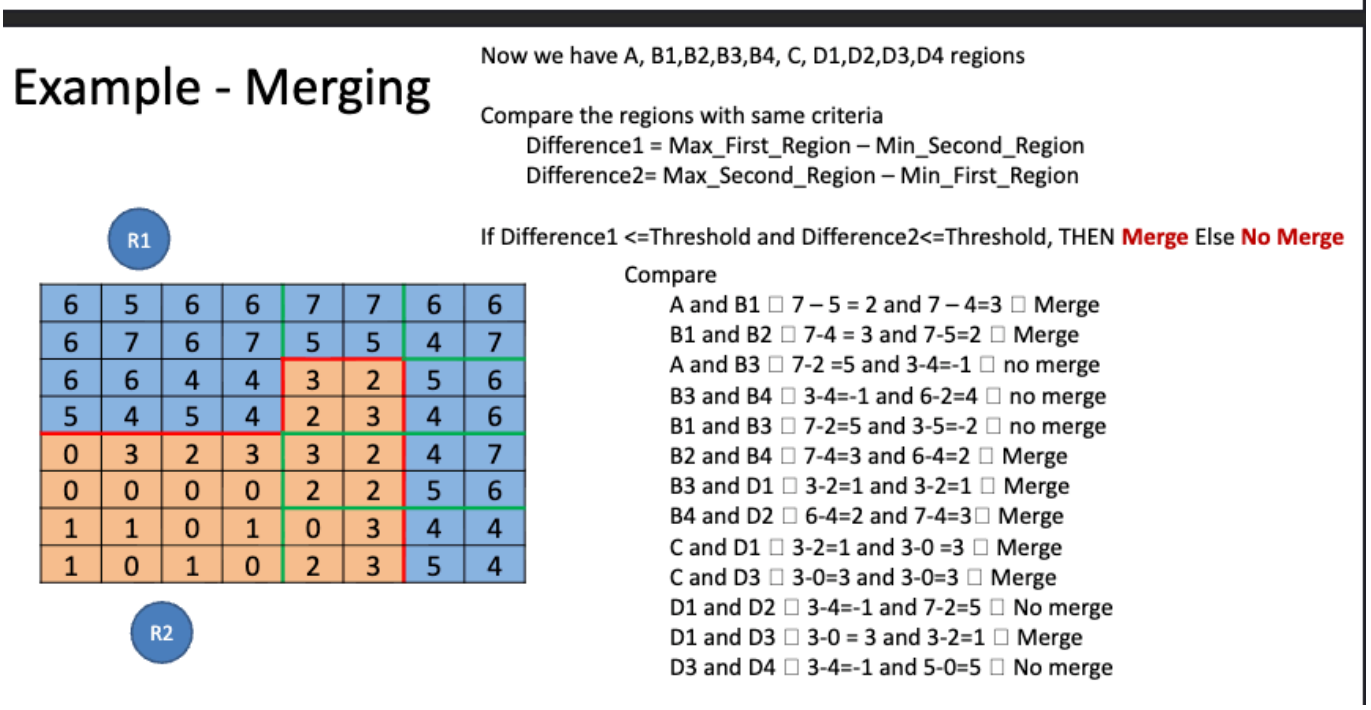
A B

6	5	6	6	7	7	6	6
6	7	6	7	5	5	4	7
6	6	4	4	3	2	5	6
5	4	5	4	2	3	4	6
0	3	2	3	3	2	4	7
0	0	0	0	2	2	5	6
1	1	0	1	0	3	4	4
1	0	1	0	2	3	5	4

C D

> After splitting, adjacent similar regions are merged — reducing over-segmentation.





> The final segmentation result after all split and merge passes.

```
[Start: Whole Image]
        ↓
[Check: Var(R) > threshold?]
   Yes → Split into 4 quadrants → repeat
   No  → Keep as leaf region
        ↓
[Merge adjacent regions if |mean(R1) − mean(R2)| < δ]
        ↓
[Stop: All regions are uniform]
```

---

b) Quadtree Representation

A **quadtree** is the data structure used by Split and Merge:

- **Root** = entire image
- **Internal node** = a region that failed homogeneity test → split into 4 quadrants
- **Leaf node** = a region that passed homogeneity test → no further splitting
- Each internal node has exactly **4 children** (4 quadrants)

```
          [Entire Image]
                |
     ┌────┬────┬────┐
   [Q1] [Q2] [Q3] [Q4]
                |
          ┌────┬────┬────┐
          [Q3a][Q3b][Q3c][Q3d]
```

Split condition: $\text{Var}(R) > \theta$
Merge condition: $|\bar{I}(R_i) - \bar{I}(R_j)| < \delta$

---

c) Advantages & Limitations

Advantages:

1. **Adaptive region sizes** — regions grow or shrink based on local image content, unlike fixed-size methods.
2. **Works on varying textures** — the variance-based split criterion naturally handles regions with different texture complexity.
3. **Simple to implement** — the quadtree structure is straightforward to build recursively.
4. **Good for structured scenes** — well-defined objects with clear intensity boundaries are segmented cleanly.

Limitations:

1. **Blocky/square boundaries** — the quadtree forces all splits to be axis-aligned rectangles, producing artificial square edges at region borders.
2. **Sensitive to threshold choice** — a poor variance threshold θ leads to either over-segmentation (too many small regions) or under-segmentation (large non-uniform regions).
3. **Computationally expensive for large images** — recursive splitting and neighbour-checking during merging can be slow on high-resolution images.
4. **May over-segment complex images** — textured or noisy regions keep failing the homogeneity test, producing unnecessarily many tiny leaf regions.

Q4 — Mean Shift Algorithm

a) Mean Shift — Definition & Steps

Mean shift clustering is a method used to find clusters in data without specifying the number of clusters beforehand. Mean shift clustering is a non-parametric, reiterative algorithm that seeks to identify groups

in a dataset by discovering the sharper peaks in a density function, when, unlike other clustering algorithms, it does not require a piece of prior knowledge on the number of categories.

Mean Shift is non-parametric = you don't need to specify the number of clusters.

Steps:

1. Place a window (circle) around a point
2. Compute the **centroid** of all points inside the window
3. **Shift** the window to that centroid
4. Repeat until the shift is smaller than threshold ε → **converged at a mode**
5. All points converging to the same mode = same cluster

```
Start → Compute centroid in window → Shift window → Repeat → Converge at
Mode
```

---

b) Mode Finding in Feature Space

Feature space is a multi-dimensional space where each dimension corresponds to a feature of the data. Every data point is represented as a vector in this space. For example, in a dataset tracking customer behavior, feature space might be two-dimensional:

Mode, in statistical terms, is the point in feature space where the data density is highest. It represents a natural cluster center.

The Concept of Mode Finding is central to mean shift clustering. The algorithm iteratively shifts data points towards the nearest mode, effectively climbing the gradient of the density function.

Mode = peak of the data density = natural cluster center.

For images, each pixel is represented in **feature space** = (color + position).

- Mean shift "climbs the hill" of density
- Pixels that converge to the **same peak** → belong to the **same segment**
- No need to pre-set number of clusters — modes are discovered automatically

---

c) Applications of Mean Shift

- **Image Segmentation** — Mean shift groups pixels by clustering them in a joint color-spatial feature space. Each pixel is represented as a vector combining its color values (RGB or Lab) and its spatial coordinates (x, y). The algorithm shifts each pixel toward the nearest density peak, and all pixels that converge to the same mode are assigned the same segment label. This produces perceptually meaningful regions without requiring a predefined number of segments, making it effective for natural scenes with smooth color gradients.

- **Object Tracking** — In video analysis, the target object's color histogram is treated as a probability distribution in feature space. The mode of this distribution corresponds to the most likely location

of the object in the current frame. Mean shift iteratively shifts a search window from the previous frame's position toward the new mode — effectively tracking the object across frames. This approach is robust to gradual appearance changes, partial occlusions, and lighting variations.

- **Clustering** — Unlike K-Means, mean shift does not require you to specify the number of clusters in advance. The algorithm automatically discovers all density peaks (modes) in the data, and each mode becomes a cluster center. Data points are assigned to the cluster whose mode they converge to. This makes mean shift suitable for exploratory data analysis where the cluster count is unknown, and for datasets where clusters have irregular shapes or varying densities.

- **Medical Imaging** — In MRI and CT scans, different tissue types (e.g., white matter, grey matter, cerebrospinal fluid, tumors) have distinct intensity and texture profiles. Mean shift clusters voxels based on these features, naturally separating tissue regions without requiring manual labeling or a fixed number of tissue classes. This is particularly useful for brain tissue segmentation, lesion detection, and quantifying tissue volumes for diagnosis or treatment monitoring.

Q5 — Graph-Based Segmentation & Normalized Cuts

a) Graph-Based Segmentation & Normalized Cut Criterion

Graph-based segmentation treats the image as a graph where pixels are nodes and edges represent similarity between pixels. The goal is to partition the graph into segments that correspond to meaningful regions in the image.

The Graph structure is defined as $G = (V, E)$ where:

- V = set of nodes (pixels)
- E = set of edges connecting nodes
- $w(u, v)$ = weight of edge between nodes u and v (similarity measure)

Segmentation goal: here the segmentation goal is to cut the graph into groups where:

- Pixels **within** a group are very similar (high edge weights)
- Pixels **between** groups are very different (low edge weights)

Normalized cut (NCut) is a criterion for evaluating the quality of a graph partition. It measures the total dissimilarity between different groups while also considering the total similarity within each group.

It prevents the algorithm from making small, isolated cuts by dividing the cut cost by the total weight of connections in each group — so both sides of the partition must be well-connected, not just small.

$$NCut(A, B) = \frac{cut(A,B)}{assoc(A,V)} + \frac{cut(A,B)}{assoc(B,V)}$$

- $cut(A,B)$ = total weight of edges crossing the cut
- $assoc(A,V)$ = total weight of all edges from group A to all nodes
- **Minimizing NCut** → balanced partitions with strong internal connections

---

b) Normalized Cut Equation (Derivation Summary)

The Normalized Cut (Ncut) equation is derived by modifying the standard graph "cut" to remove the bias toward small, isolated segments. The goal is to define a measure that balances dissimilarity between groups with similarity within groups.

Step 1 — Standard Cut

For a graph $G = (V, E)$ partitioned into two disjoint sets A and B, the cut is the sum of weights of all edges connecting the two sets:

$$cut(A, B) = \sum_{u \in A,\ v \in B} w(u, v)$$

Minimizing this value alone leads to unbalanced cuts — a single node with fewer edges can be cheaply isolated, which is not a useful segmentation.

Step 2 — Normalized Cut

To fix this, the cut cost is normalized by the total connection weight (the "volume" or "association") of each segment to the entire graph:

$$NCut(A, B) = \frac{cut(A,B)}{assoc(A,V)} + \frac{cut(A,B)}{assoc(B,V)}$$

where:

- $cut(A,B)$ = total weight of edges crossing the cut
- $assoc(A,V) = \sum_{u \in A,\ v \in V} w(u,v)$ = total weight of all edges from group A to the whole graph

Step 3 — Relation to Normalized Association (Nasso)

The "goodness" of a partition can also be measured by how tightly connected nodes are within each segment — called **Normalized Association**:

$$Nasso(A, B) = \frac{assoc(A,A)}{assoc(A,V)} + \frac{assoc(B,B)}{assoc(B,V)}$$

Since $assoc(A, V) = assoc(A, A) + cut(A, B)$, substituting into the NCut formula:

$$NCut(A, B) = \frac{assoc(A,V) - assoc(A,A)}{assoc(A,V)} + \frac{assoc(B,V) - assoc(B,B)}{assoc(B,V)}$$

$$NCut(A, B) = \left(1 - \frac{assoc(A,A)}{assoc(A,V)}\right) + \left(1 - \frac{assoc(B,B)}{assoc(B,V)}\right)$$

$$\boxed{NCut(A, B) = 2 - Nasso(A, B)}$$

> This proves that **minimizing NCut** (disassociation between segments) is exactly equivalent to **maximizing Nasso** (association within segments) — the two objectives are the same problem. Minimizing NCut leads to a **generalized eigenvalue problem** solved via the graph Laplacian matrix.

---

c) Normalized Cut vs Minimum Cut

Minimum Cut	Normalized Cut
Cuts cheapest edges	Balanced partition
Often isolates single nodes	Considers whole group connectivity
Biased toward small regions	Fair to both halves
Simple but poor results	Better perceptual segmentation

Q6 — Graph Cuts for Image Segmentation

a) Graph Cuts for Segmentation

An image is encoded as an undirected graph $G = (V, E)$ where:

- V contains one **vertex for each pixel**
- E contains an **edge between pixels i and j** if i and j are neighbors (4-connected or 8-connected grid)

Image Segmentation Problem:

We add parameters to model the separation of neighboring pixels. For every neighboring pair ${i, j}$, we define:

- p_{ij} = penalty for placing one of i, j in foreground and the other in background

The goal is to **partition pixels into two sets A (foreground) and B (background)** to maximize:

$$q(A, B) = \sum_{i \in A} a_i + \sum_{j \in B} b_j - \sum_{\substack{(i,j) \in E \ i,j \text{ separated}}} p_{ij}$$

where:

- a_i = likelihood that pixel i is in the **foreground**
- b_i = likelihood that pixel i is in the **background**
- p_{ij} = boundary penalty for separating neighboring pixels i and j

The first two terms reward pixels being assigned to their most likely label; the third term penalizes cuts that split strongly connected neighbors — encouraging smooth boundaries.

To represent this as a graph cut problem, we introduce two special nodes:

- **Source (S)** = foreground label
- **Sink (T)** = background label

Every pixel gets connected to both S and T. The algorithm finds the **minimum cut** that separates foreground from background.

Cut = set of edges removed to disconnect S from T.
Minimum cut = cut with lowest total weight = optimal segmentation boundary.

---

b) Energy Minimization Framework

Energy-based segmentation methods frame the labeling problem as an **optimization task**: find the assignment of labels to pixels that minimizes a total energy (cost) function. The energy function is designed with two competing objectives:

- **Data fidelity** — how well the assigned label (foreground/background) matches what the image evidence suggests for that pixel (e.g., based on color histograms or intensity models)
- **Smoothness** — how regular the segmentation boundary is; neighboring pixels with different labels incur a penalty, discouraging jagged or noisy boundaries

This is formalized as a **Markov Random Field (MRF)**, where each pixel's label depends only on its neighbors. The total energy to minimize is:

$$E(\mathbf{x}) = \underbrace{\sum_i D_i(x_i)}{|text{data term}} + |underbrace{|sum{(i,j) \in \mathcal{N}} V_{ij}(x_i, x_j)}_{\text{smoothness term}}$$

Data term $D_i(x_i)$: The cost of assigning label x_i (foreground or background) to pixel i. This is derived from a color/intensity model — if pixel i strongly matches the foreground color model, $D_i(\text{foreground})$ is low.

Smoothness term $V_{ij}(x_i, x_j)$: The penalty for assigning different labels to neighboring pixels i and j. Typically defined as:

$$V_{ij}(x_i, x_j) = \begin{cases} 0 & \text{if } x_i = x_j \ w_{ij} & \text{if } x_i \neq x_j \end{cases}$$

where w_{ij} is high when pixels i and j are similar (discouraging cuts between similar pixels) and low when they are dissimilar (allowing cuts at natural boundaries).

Goal: Find the labeling $\mathbf{x}^* = \arg\min E(\mathbf{x})$ — the assignment that best fits the image data while keeping the boundary smooth. Graph cuts solve this exactly and efficiently via the max-flow min-cut theorem.

---

c) Max-Flow Min-Cut Theorem

Theorem: The maximum flow that can be sent from Source (S) to Sink (T) in a flow network equals the capacity of the minimum cut separating S from T.

$$\text{Max-Flow}(S \to T) = \text{Min-Cut capacity}$$

Concept — Flow Network Analogy:

The image graph is modeled as a flow network where each edge is treated as a pipe with a capacity equal to its edge weight. Flow is pushed from the Source (S = foreground) to the Sink (T = background). The maximum amount of flow that can travel from S to T is limited by a set of bottleneck edges — these bottleneck edges form the minimum cut.

Why this solves segmentation:

When max-flow is computed, the edges that are fully saturated (carrying flow equal to their capacity) and cannot pass any more flow form the minimum cut. Removing these edges disconnects S from T,

partitioning the image into two regions:

- All nodes reachable from S → **foreground**
- All nodes reachable from T → **background**

The minimum cut naturally falls along weak edges (low similarity between pixels) — which correspond to object boundaries — and avoids cutting strong edges (highly similar neighboring pixels inside a region).

Connection to Energy Minimization:

Minimizing the graph cut capacity is equivalent to minimizing the MRF (Markov Random Field) energy function. The data term D_i is encoded in the edge weights between pixels and the S/T terminals, and the smoothness term V_{ij} is encoded in the edges between neighboring pixels. Solving max-flow therefore simultaneously minimizes both terms optimally.

Algorithm (Ford-Fulkerson / Push-Relabel):

1. Initialize flow on all edges to 0
2. Find an **augmenting path** from S to T (a path with remaining capacity)
3. Push flow along this path equal to the bottleneck capacity
4. Repeat until no augmenting path exists
5. The saturated edges crossing from S-side to T-side form the **minimum cut = segmentation boundary**

> **Max-flow = Min-cut** → solving max-flow gives the globally optimal foreground/background segmentation in polynomial time.

Q7 — 2D Feature Detection, Matching & Alignment

a) Feature Detection & Matching in 2D

Definition: Feature detection is the process of finding distinctive, repeatable points or regions in an image — called **keypoints** — that can be reliably located even when the image is scaled, rotated, or has changed lighting. Feature matching then finds corresponding keypoints across two or more images to understand the geometric relationship between them.

Why Features? Comparing images pixel-by-pixel is fragile and slow. Instead, a small set of distinctive keypoints (corners, blobs, edges) captures the structure of an image compactly and can be matched across views robustly.

Step 1 — Feature Detection

A detector scans the image for points with strong, unique local structure. Common detectors:

- **Harris Corner Detector** — finds corners by looking for large intensity changes in all directions. A corner has high gradient magnitude in multiple directions.
- **SIFT (Scale-Invariant Feature Transform)** — detects blob-like regions at multiple scales using a Difference-of-Gaussian (DoG) pyramid. Invariant to scale and rotation.

- **SURF (Speeded-Up Robust Features)** — faster approximation of SIFT using integral images and Haar wavelets.
- **ORB (Oriented FAST and Rotated BRIEF)** — real-time detector/descriptor combining FAST keypoints with BRIEF descriptors. Used in mobile and embedded systems.

Step 2 — Feature Description

Each detected keypoint is described by a **descriptor** — a numerical vector that summarizes the local image appearance around the keypoint. A good descriptor is invariant to rotation, scale, and illumination changes.

- SIFT descriptor: 128-dimensional vector of gradient orientations in a 16×16 patch
- ORB descriptor: 256-bit binary string (much faster to compute and compare)

Step 3 — Feature Matching

Descriptors from two images are compared to find corresponding keypoints. The most common strategy is **nearest-neighbor matching**: for each keypoint in image 1, find the keypoint in image 2 whose descriptor is closest (using Euclidean distance for SIFT, Hamming distance for ORB).

To reject ambiguous matches, **Lowe's ratio test** is applied: $$\frac{d_1}{d_2} < 0.75$$ A match is accepted only if the best match is significantly closer than the second-best.

Step 4 — Outlier Rejection with RANSAC

Even after ratio testing, some matches are wrong (outliers). **RANSAC (Random Sample Consensus)** robustly estimates the transformation by:

1. Randomly sampling a minimal set of matches (e.g., 4 for homography)
2. Computing the transformation from that sample
3. Counting **inliers** — matches consistent with this transformation (reprojection error < threshold)
4. Repeating many times and keeping the transformation with the most inliers

The final transformation is re-estimated using all inliers.

---

b) Transformation Models

When two images of the same scene are taken from different viewpoints or with different camera settings, a **geometric transformation** relates the coordinates of one image to the other. These transformations differ in the number of degrees of freedom (DOF) and in what geometric properties they preserve.

---

1. Translation (2 DOF)

The simplest transformation — shifts every point by a fixed amount:

$$x' = x + t_x, \quad y' = y + t_y$$

$$\begin{bmatrix} x' \ y' \end{bmatrix} = \begin{bmatrix} x \ y \end{bmatrix} + \begin{bmatrix} t_x \ t_y \end{bmatrix}$$

- Preserves: shape, size, orientation, parallel lines, distances
- Use case: aligning images taken from the same viewpoint with a slight shift (e.g., video stabilization)

---

2. Rotation (1 DOF)

Rotates the image by angle θ around the origin:

$$x' = x\cos\theta - y\sin\theta, \quad y' = x\sin\theta + y\cos\theta$$

$$\begin{bmatrix} x' \ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \ y \end{bmatrix}$$

- Preserves: shape, size, distances (isometry)
- Use case: correcting camera tilt

---

3. Rigid Transform (3 DOF = Rotation + Translation)

Combines rotation and translation. Preserves all distances and angles:

$$\begin{bmatrix} x' \ y' \end{bmatrix} = R \begin{bmatrix} x \ y \end{bmatrix} + \begin{bmatrix} t_x \ t_y \end{bmatrix}$$

- Preserves: distances, angles, shape
- Use case: aligning scans of the same object from slightly different positions

---

4. Affine Transform (6 DOF)

A general linear transformation followed by translation. It can represent rotation, scale, shear, and reflection simultaneously:

$$\begin{bmatrix} x' \ y' \end{bmatrix} = \begin{bmatrix} a & b \ c & d \end{bmatrix} \begin{bmatrix} x \ y \end{bmatrix} + \begin{bmatrix} t_x \ t_y \end{bmatrix}$$

- Preserves: **parallel lines** (parallel lines remain parallel after transformation)
- Does NOT preserve: distances, angles
- Use case: correcting camera tilt + zoom + slight skew; satellite image alignment

Requires at least **3 point correspondences** to solve (6 unknowns, 2 equations per point).

---

5. Homography (8 DOF)

The most general 2D-to-2D transformation for planar scenes, also called a **projective transform**:

$$\begin{bmatrix} x' \ y' \ 1 \end{bmatrix} \sim \begin{bmatrix} h_{11} & h_{12} & h_{13} \ h_{21} & h_{22} & h_{23} \ h_{31} & h_{32} & 1 \end{bmatrix} \begin{bmatrix} x \ y \ 1 \end{bmatrix}$$

- Preserves: straight lines (but NOT parallel lines)
- Models the full perspective change when the camera moves around a planar scene
- Requires at least **4 point correspondences**

- Use case: image stitching, AR marker tracking, document scanning

Transform	DOF	Preserves
Translation	2	Distances, angles, shape
Rotation	1	Distances, angles
Rigid	3	Distances, angles
Affine	6	Parallel lines
Homography	8	Straight lines only

---

c) Applications in Image Stitching

What is Image Stitching?

Image stitching is the process of combining multiple photographs with overlapping fields of view into a single wide-angle or panoramic image. It is used when a single image cannot capture the full scene.

How Feature Matching and Transformation Models are Used:

Step 1 — Capture overlapping images Two or more photos are taken with overlapping regions (typically 20–40% overlap) so features can be found in common between them.

Step 2 — Detect and describe features SIFT or ORB detects keypoints in each image and computes descriptors for each keypoint.

Step 3 — Match features across images Descriptors are matched between image pairs. Only matches in the overlapping region are correct; RANSAC filters out wrong matches.

Step 4 — Estimate homography Using the inlier matches, a homography matrix H is computed that maps the coordinates of one image onto the coordinate system of the other: $$p_2 \sim H \cdot p_1$$

Step 5 — Warp images One image is geometrically warped (transformed) using the homography so that it aligns with the other. This is done using inverse warping + bilinear interpolation to avoid holes.

Step 6 — Blend the seam The overlapping regions are blended (e.g., using linear blending or multi-band blending) to hide the seam and produce a seamless panorama.

Real-World Uses:

- **Google Street View** — 360° panoramas stitched from multiple camera feeds
- **Drone mapping** — aerial images stitched into orthomosaic maps for agriculture and surveying
- **Medical imaging** — stitching microscope images to produce a full-slide view
- **Virtual tours** — real estate and museum virtual tours use stitched panoramic photography
- **Satellite imagery** — multiple satellite passes stitched to build large-area maps

---

Q8 — 3D Point Cloud Alignment & ICP

a) 3D Point Cloud Alignment

A **point cloud** is a collection of points in 3D space, where each point represents a location on a physical surface. Point clouds are captured using sensors like **LiDAR** (Light Detection and Ranging), **depth cameras** (e.g., Microsoft Kinect), or **stereo cameras**. Each point has coordinates (x, y, z) and sometimes additional attributes like color or surface normal.

When the same object or scene is scanned from different positions or angles, each scan produces a separate point cloud. These clouds overlap in 3D space but are not aligned — they are in different coordinate systems. **Point cloud alignment** (also called **registration**) is the process of finding the rigid transformation (rotation + translation) that maps one point cloud onto the other so they represent the same geometry in a common coordinate frame.

$$P' = R \cdot P + t$$

where R is the rotation matrix and t is the translation vector that best aligns the source cloud P to the target cloud P'.

---

b) ICP (Iterative Closest Point) Algorithm

ICP is the most widely used algorithm for aligning two point clouds when no known feature correspondences exist. The core idea is simple: repeatedly find which point in the target cloud is closest to each point in the source cloud, compute the best transformation to reduce that distance, apply it, and repeat until the clouds are aligned.

Steps:

1. **Initialize** — Start with an initial guess for the alignment (often the identity transform, or a rough manual alignment). A good starting point speeds up convergence.
2. **Find Correspondences** — For each point in the source cloud, find its nearest neighbor in the target cloud. These pairs are the assumed correspondences for this iteration.
3. **Estimate Transform** — Compute the rotation R and translation t that minimizes the total squared distance between all correspondence pairs. This is solved exactly using **SVD (Singular Value Decomposition)**.
4. **Apply Transform** — Move the source cloud using the computed R and t. The source cloud is now closer to the target than before.
5. **Check Convergence** — Measure how much the cloud moved. If the change is smaller than a threshold (i.e., the clouds are close enough), stop. Otherwise go back to step 2 with the updated source cloud.

Key limitation: ICP always converges, but it may converge to a **local minimum** — a wrong alignment that looks locally optimal. This is why a good initial alignment matters. Techniques like **random restarts** or **coarse-to-fine alignment** are used to avoid this.

---

c) Applications

- **3D Reconstruction** — When scanning an object or room from multiple angles (e.g., with a handheld depth camera), each scan covers only a portion of the surface. ICP aligns all partial scans together into a single complete 3D model. This is used in heritage preservation, reverse engineering, and VFX.

- **Autonomous Vehicles** — Self-driving cars use LiDAR to build a map of the environment. As the car moves, each new LiDAR scan must be aligned with the previous ones to track the vehicle's position and update the map. ICP (or its variants) performs this real-time scan matching.
- **Robotics — SLAM** — In Simultaneous Localization and Mapping, a robot builds a map of an unknown environment while tracking its own location within it. ICP aligns successive point cloud scans to estimate how far the robot has moved and in what direction.
- **Medical Imaging** — Surgeons sometimes need to align a pre-operative 3D scan (e.g., MRI or CT) with an intra-operative scan or physical tracker. ICP registers these two 3D datasets so that surgical tools can be accurately guided to the correct anatomical location.

Q9 — Pose Estimation

a) Pose Estimation — Definition & Importance

Pose estimation is the task of determining the **position and orientation** of an object (or camera) in 3D space. An object's pose has **6 degrees of freedom (DOF)**: 3 for rotation (how it is tilted/rotated along each axis) and 3 for translation (where it is in space along X, Y, Z).

In simpler terms: given a photo of an object, pose estimation answers — *where exactly is this object, and which direction is it facing, relative to the camera?*

Why it matters:

- **Robotics** — A robotic arm needs to know the exact position and orientation of an object to grasp it correctly. Grabbing a cup requires knowing both where the cup is and which way its handle is pointing.
- **Augmented Reality (AR/VR)** — Virtual objects must be placed at precisely the right position and angle in the real-world scene. If the pose is slightly wrong, virtual furniture appears to float or clip through the floor.
- **Autonomous Vehicles** — The car must estimate the orientation of other vehicles and pedestrians to predict their movement and plan a safe path around them.
- **Face Recognition** — Before comparing faces, the head pose (looking left, right, down) must be estimated to normalize the face to a frontal view.

---

b) Perspective-n-Point (PnP) Problem

The **PnP problem** is the standard mathematical formulation of pose estimation. The setup is:

You already know the 3D coordinates of certain points on the object (from a CAD model or prior measurement). You also know where those same points appear as 2D pixels in a photo. Given this set of 3D–2D correspondences, PnP finds the rotation and translation of the camera (or equivalently the object) that explains all the projections simultaneously.

$$p_i = K [R | t] P_i$$

where K is the camera's intrinsic matrix (lens properties like focal length), R and t are the rotation and translation to find, P_i is a 3D world point, and p_i is its observed 2D image projection.

Methods — how many point pairs are needed:

- **P3P** — Uses the minimum possible: just 3 point correspondences. Fast, but gives up to 4 ambiguous solutions; a 4th point is used to pick the correct one. Used when speed is critical.
- **EPnP (Efficient PnP)** — Works with 4 or more points. Expresses all 3D points as weighted combinations of 4 virtual control points, making the algebra much simpler. Runs in near-linear time regardless of how many points are used.
- **Iterative PnP** — Starts from an initial estimate (e.g., EPnP result) and refines it using non-linear optimization, minimizing the **reprojection error** — the pixel distance between the observed and projected points. Most accurate, but slower.

In practice: OpenCV's `solvePnP()` handles all of this automatically, choosing the right method based on the number of points provided.

---

c) Applications

Application	Use
AR/VR	Place virtual objects in correct real-world position
Robotics	Grasp objects at correct angle
Autonomous Driving	Detect vehicle orientation, pedestrian pose
Face Recognition	Head pose for face alignment

Q10 — Comparison of All Segmentation Techniques

a) Snakes vs Level Sets vs Graph Cuts vs Mean Shift

Snakes (Active Contours) work by placing a curve on the image and letting it deform under two forces: an internal force that keeps the curve smooth, and an external force that pulls it toward edges. The curve is represented explicitly as a set of connected points. Because of this, Snakes cannot handle topology changes — if the object splits or has a hole, the snake cannot split or merge accordingly. They require the user to place the initial curve reasonably close to the object boundary, so they are semi-automatic. Snakes work best when the target boundary is smooth and you already have a rough idea of where it is.

Level Sets solve the topology problem by representing the contour implicitly — as the zero-level of a function defined over the entire image. When the shape splits or merges, the function simply updates automatically without any special handling. This makes level sets far more flexible for complex or changing shapes. The tradeoff is computational cost — updating the function across the whole image is slower than moving a curve. Like Snakes, level sets also require the user to set an initial region, making them semi-automatic. They extend naturally to 3D because the same implicit representation works in any number of dimensions.

Graph Cuts represent the image as a graph where every pixel is a node and edges connect neighboring pixels. Segmentation is posed as finding the minimum-cost cut that separates a foreground group of

nodes from a background group. The user provides **seed points** — a few pixels manually marked as foreground and background — and the algorithm finds the globally optimal boundary from those seeds. Because it uses max-flow, it is fast and gives a guaranteed global optimum (not just a local one). Graph Cuts can handle any shape but needs user seeds to define what foreground and background mean.

Mean Shift is the most automatic of the four — it requires no user input at all. It works by finding regions of high pixel density in a color or feature space. Each pixel is assigned to the nearest density peak (called a mode), and all pixels belonging to the same peak form a segment. Mean Shift is particularly good at segmenting images based on color similarity, and it naturally discovers the number of segments rather than requiring you to specify it. The downside is that it is slower than Graph Cuts, and the result depends heavily on the bandwidth parameter that controls the size of the search window.

In summary: Use Snakes when you have a smooth known boundary shape. Use Level Sets when the shape is complex or may change topology. Use Graph Cuts when you need fast, globally optimal foreground/background separation and can provide seeds. Use Mean Shift when you want fully automatic color-based segmentation without any prior knowledge.

---

b) How Segmentation Helps in Feature Alignment & Pose Estimation

Segmentation → Feature Alignment:

- Segmentation isolates the object of interest → reduces false feature matches
- Only features within the segmented region are used for alignment → more accurate homography/transform estimation
- Example: Segment a building from background, then match features only on the building for stitching

Segmentation → Pose Estimation:

- Segment the object first → extract its 2D boundary/keypoints
- Use those keypoints as the ${p_i}$ correspondences for PnP
- Without segmentation, background clutter causes wrong correspondences → wrong pose
- Example: Segment a car → detect corners → estimate 3D pose of the car

> **Pipeline:** Segment → Detect features → Match → Estimate transformation/pose

---

Problems — Solutions

Problem 1: Energy Function in Snakes

Given: 5 points: $(1,2), (2,3), (3,5), (4,4), (5,3)$
$\alpha = 0.5$, $\beta = 0.3$, External energy = $[2, 1, 3, 2, 1]$

Step 1: Elasticity (first derivative — distance between consecutive points)

$$E_{elastic} = \alpha \sum_{i=2}^{5} \sqrt{(x_i - x_{i-1})^2 + (y_i - y_{i-1})^2}$$

Pair	Distance
$(1,2)→(2,3)$	$\sqrt{1+1} = \sqrt{2} \approx 1.41$
$(2,3)→(3,5)$	$\sqrt{1+4} = \sqrt{5} \approx 2.24$
$(3,5)→(4,4)$	$\sqrt{1+1} = \sqrt{2} \approx 1.41$
$(4,4)→(5,3)$	$\sqrt{1+1} = \sqrt{2} \approx 1.41$

Sum $= 1.41 + 2.24 + 1.41 + 1.41 = 6.47$
$E_{elastic} = 0.5 \times 6.47 = \mathbf{3.24}$

Step 2: Rigidity (second derivative — curvature)

$$E_{rigid} = \beta \sum_{i=2}^{4} \sqrt{(x_{i+1}-2x_i+x_{i-1})^2 + (y_{i+1}-2y_i+y_{i-1})^2}$$

Triple	Curvature
$(1,2),(2,3),(3,5)$	$
$(2,3),(3,5),(4,4)$	$
$(3,5),(4,4),(5,3)$	$

Sum $= 1 + 3 + 0 = 4$
$E_{rigid} = 0.3 \times 4 = \mathbf{1.2}$

Step 3: Internal Energy $$E_{internal} = E_{elastic} + E_{rigid} = 3.24 + 1.2 = \mathbf{4.44}$$

Step 4: External Energy $$E_{external} = -(2 + 1 + 3 + 2 + 1) = \mathbf{-9}$$

Step 5: Total Energy $$E_{snake} = E_{internal} + E_{external} = 4.44 + (-9) = \mathbf{-4.56}$$

Problem 2: Split and Merge

Given: Variance threshold $\theta = 10$

Part a) Which regions split?

Region	Variance	Split?
R1	15	**Yes** (15 > 10)
R2	8	No (8 < 10)

→ **R1 is split** into 4 subregions.

Part b) Subregions of R1 = [5, 6, 12, 4]

Subregion	Variance	Split?
R1a	5	No

Subregion	Variance	Split?
R1b	6	No
R1c	12	**Yes** (12 > 10)
R1d	4	No

→ **R1c is split further.**

Problem 3: Max-Flow Min-Cut

Given graph:

```
S → A = 10
S → B = 5
A → B = 15
A → T = 10
B → T = 10
```

Each edge has a **capacity** — think of it as a pipe that can carry at most that much flow. Flow starts at the Source (S) and must reach the Sink (T). The goal is to find the maximum total flow that can travel from S to T without exceeding any pipe's capacity.

Part a) Maximum Flow from S to T

To find the max flow, we look for all distinct paths from S to T and see how much flow each can carry. The flow along any path is limited by its **bottleneck** — the edge with the smallest capacity on that path.

Path 1: S → A → T

The edges on this path are S→A (capacity 10) and A→T (capacity 10). The bottleneck is min(10, 10) = **10**. So we can push 10 units of flow along this path. After doing this, both edges S→A and A→T are fully used (saturated).

Path 2: S → B → T

The edges on this path are S→B (capacity 5) and B→T (capacity 10). The bottleneck is min(5, 10) = **5**. So we can push 5 units of flow along this path. After doing this, S→B is fully saturated. B→T still has 5 units of remaining capacity.

Can we find any more paths?

After the two paths above, S→A is saturated (10/10) and S→B is saturated (5/5). There is no more flow that can leave S — both outgoing edges are full. So no more augmenting paths exist.

$$\text{Total Max Flow} = 10 + 5 = \mathbf{15}$$

Part b) Minimum Cut

A **cut** is a set of edges whose removal disconnects S from T completely. The **minimum cut** is the cut with the smallest total capacity. By the Max-Flow Min-Cut theorem, the minimum cut capacity always equals the maximum flow.

To find the minimum cut, we identify which edges are fully saturated (bottlenecks that stopped us from pushing more flow):

- **S → A = 10** — fully saturated (10/10 used) ✓
- **S → B = 5** — fully saturated (5/5 used) ✓

If we remove these two edges, there is no way to get any flow from S to T at all — S becomes completely isolated. So the minimum cut is the set ${S \to A,\ S \to B}$.

$$\text{Min-Cut capacity} = 10 + 5 = \mathbf{15}$$

> **Min-Cut = Max-Flow = 15** ✓
> This confirms the Max-Flow Min-Cut theorem: the maximum flow equals the capacity of the minimum cut. In image segmentation, this minimum cut corresponds to the optimal boundary between foreground and background.

Quick Revision Cheat Sheet

Topic	Key Formula / Idea
Snakes	$E = \alpha\|C'\|^2 + \beta\|C''\|^2 - \|\nabla I\|^2$
Level Sets	Contour = zero of $\phi(x,t)$; evolves via PDE
Intelligent Scissors	Dijkstra on pixel graph; high gradient = low cost
Split & Merge	Var > θ → split;
Mean Shift	Slide window to centroid until converged → mode
Normalized Cut	$NCut = \frac{cut}{assoc(A)} + \frac{cut}{assoc(B)}$
Graph Cuts	Min-cut = segmentation boundary; Max-flow = Min-cut
ICP	Match → Fit → Transform → Repeat until aligned
PnP Problem	Given 3D-2D pairs, find R and t